

0.32. Le trajet dans un tableau

Les pas du trajet rangés dans deux tableaux : une boucle `for` les parcourt. On change le chemin sans toucher au code.

1. Ce qu'il faut comprendre

Le carré du 0.27 répétait quatre fois la même ligne, avec d'autres nombres. Quand un programme répète la même chose avec des nombres différents, on range les **nombres** dans un tableau, et on écrit la ligne **une seule fois**, dans une boucle.

Ce qui est nouveau ici : deux tableaux, `PAS_X` et `PAS_Y`. La case `i` de chacun dit le pas du côté `i` : `PAS_X[0]` et `PAS_Y[0]` pour le premier côté (5 et 0 : à droite), `PAS_X[1]` et `PAS_Y[1]` pour le deuxième (0 et 5 : en bas), et ainsi de suite.

Des nombres négatifs dans un tableau d'octets : `-5` y est rangé comme 251, comme toujours. `deplace` sait le lire : au-delà de 127, c'est un pas en arrière.

`const` devant le tableau veut dire qu'il ne change jamais : il est gravé dans la cartouche, et ne prend pas de place dans la mémoire de travail.

La boucle `for` fait tourner `i` de 0 à 3 : `sizeof(PAS_X)` vaut 4, le nombre de cases (comme `sizeof(ALPHABET)` au 0.9). Et la boucle `while (true)` autour recommence le trajet sans fin.

Pour changer le chemin, on ne touche qu'aux tableaux. Essaie `{ 3, 3, 3, -9 }` et `{ 3, -3, 3, 0 }` : un zigzag. Ajoute des cases, la boucle suit toute seule.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace>` → [0.25.3. La fonction deplace\(\) — avancer sur les deux axes](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1195 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      62 o ← la console, à l'allumage — la mise en route de la console
deplace()      236 o ← main() — une fonction fournie par le langage
main()         67 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() — efface des cases
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles — la police entière
```

```

PAS_X_0      4 o ← main()  - une table gravée
PAS_Y_1      4 o ← main()  - une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()   × 2    26 o sur place, dans deplace() + AdresseCase (10 o)
  effacer() × 1    15 o sur place, dans deplace() + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 1     3 o sur place, dans deplace() + AttendreImage (28 o)
Tes fonctions appelées :
  deplace() × 1    39 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// Le trajet : un côté par case. PAS_X[i] et PAS_Y[i] vont ensemble.
//
//      côté 0   côté 1   côté 2   côté 3
//      droite  bas    gauche  haut

#include <deplace>    // fait avancer une tuile sur les deux axes
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A

```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PAS_X

```
const uint8_t PAS_X[] = { 5,      0,      -5,      0 };
```

3. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PAS_Y

```
const uint8_t PAS_Y[] = { 0,      5,      0,      -5 };
```

4. UNE VARIABLE — X

```
uint8_t x = 0;      // la colonne de la lettre
```

5. UNE VARIABLE — Y

```
uint8_t y = 0;      // la ligne de la lettre
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
  while (true) {
    // le trajet, sans fin
    for (uint8_t i = 0; i < sizeof(PAS_X); i++) { // i = 0, 1, 2, 3 : un tour par côté
      // Le côté i : on lit ses deux pas dans les tableaux.
      // i = 0 : deplace(x, y, ALPHABET[0], 5, 0); à droite
      // i = 1 : deplace(x, y, ALPHABET[0], 0, 5); en bas
      // i = 2 : deplace(x, y, ALPHABET[0], -5, 0); à gauche
      // i = 3 : deplace(x, y, ALPHABET[0], 0, -5); en haut
      deplace(x, y, ALPHABET[0], PAS_X[i], PAS_Y[i]);
    }
  }
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



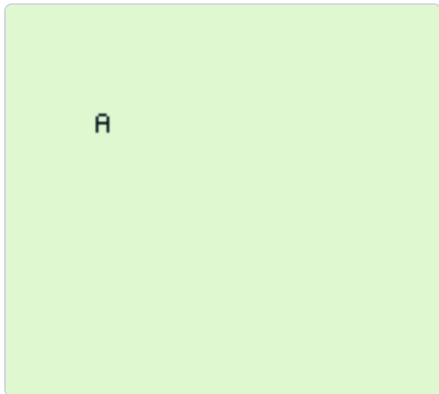
la toute première image — la console vient de démarrer — image 1



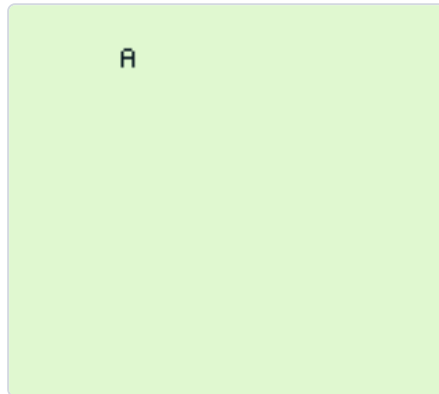
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui tourne en carré sans s'arrêter : droite, bas, gauche, haut, et on recommence.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les quatre coins, dans l'ordre (5,0 5,5 0,5 0,0)

✓ et il recommence

✓ un seul A à l'écran

Ensuite : **0.32.1. Le trajet dans un tableau — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++